Light

Programming – Data Structures

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Variables, constants and data types
2. 1D lists (arrays)
3. 2D lists and records
Worked examples
Quick reference
Typical pitfalls

Edexcel GCSE Computer Science — **Programming Data Structures (1CP2/02 Topic 6: Problem Solving with Programming)**. This topic covers how data is stored and organised in programs, including variables, constants, data types, 1D lists (arrays), 2D lists, and records.

Paper 2 questions require you to **declare** variables and data structures, **use** appropriate data types, **manipulate** lists using indexing and built-in methods, and **process** 2D structures and records. Understanding data structures is essential for solving programming problems efficiently.

Why data structures matter

Data structures are ways of organising and storing data so it can be accessed and modified efficiently. Choosing the right data structure is fundamental to good programming:

- **Organisation:** Group related data together logically
- **Efficiency:** Access and process data quickly

- **Clarity:** Make code easier to read and maintain
- **Flexibility:** Handle varying amounts of data
- **Exam relevance:** Data structures appear in almost every Paper 2 question — you will need to declare, populate, and process lists in most tasks

What you must know

This checklist covers the key exam-visible concepts from Edexcel 1CP2 Topic 6 relating to data structures.

- **Declare** variables with meaningful identifiers and assign values to them
- **Use** constants for values that should not change during program execution
- **Select** appropriate data types: integer, real (float), Boolean, character, and string
- **Convert** between data types using int(), float(), str(), and bool()
- **Create** 1D lists and access elements using index notation
- **Use** list methods: append(), insert(), remove(), pop(), and len()
- **Create** and access elements in 2D lists using double index notation
- **Use** records (dictionaries) to store related data with named fields
- **Iterate** through lists and records using for loops

> **Specification link:** This topic covers Edexcel 1CP2 Paper 2 Topic 6.2 (Data types, variables, constants) and Topic 6.3 (Data structures: 1D arrays, 2D arrays, records). All code must use Python 3 within the Programming Language Subset (PLS).

Toolkit

These are the key patterns, syntax templates, and code structures you need for data structure questions in Paper 2.

Data types summary

Data type	Description	Python example	Use case
Integer	Whole numbers (positive or negative)	`age = 16`	Counting, indexing, IDs
Real (float)	Numbers with decimal places	`price = 9.99`	Prices, measurements, averages
Boolean	True or False only	`found = True`	Flags, conditions, yes/no
Character	Single letter, digit, or symbol	`grade = "A"`	Grades, initials, single symbols
String	Sequence of characters (text)	`name = "Alice"`	Names, messages, text data

Variable and constant patterns

Variables

```
# Declaration and assignment
score = 0
name = "Player1"
is_active = True

# Updating variables
score = score + 10
name = input("Enter name: ")
```

Rule: Use lowercase with underscores for variable names (snake_case).

Constants

```
# Constants use UPPERCASE
MAX_LIVES = 3
TAX_RATE = 0.2
GREETING = "Welcome"

# Constants should not be changed
# during program execution
```

Rule: Use UPPERCASE with underscores for constants.

Type conversion functions

Function	Converts to	Example	Result
`int()`	Integer	`int("42")`	42
`int()`	Integer (truncates)	`int(3.9)`	3
`float()`	Float/Real	`float("3.14")`	3.14
`str()`	String	`str(100)`	"100"
`bool()`	Boolean	`bool(1)`	True

1D list operations

Creating and accessing lists

```
# Create empty list
scores = []

# Create with values
```

```
names = ["Ali", "Ben", "Cat"]

# Access by index (starts at 0)
first = names[0]    # "Ali"
last = names[2]     # "Cat"
last = names[-1]    # "Cat" (negative index)

# Get length
size = len(names)   # 3
```

List methods

```
# Add to end
names.append("Dan")

# Insert at position
names.insert(1, "Eve")

# Remove by value
names.remove("Ben")

# Remove by index (returns value)
item = names.pop(0)

# Remove last item
last = names.pop()
```

2D list patterns

```
# Create 2D list (list of lists)
grid = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Access: grid[row][column]
value = grid[0][1]    # Row 0, Column 1 = 2
value = grid[2][0]    # Row 2, Column 0 = 7
```

```
# Iterate through 2D list
for row in grid:
    for item in row:
        print(item)
```

Record (dictionary) patterns

```
# Create a record (dictionary)
student = {
    "name": "Alice",
    "age": 16,
    "grade": "A"
}

# Access fields
print(student["name"])    # "Alice"
print(student["age"])     # 16

# Update a field
student["age"] = 17

# Add a new field
student["school"] = "High School"

# List of records
students = [
    {"name": "Alice", "age": 16},
    {"name": "Bob", "age": 15},
    {"name": "Charlie", "age": 16}
]

# Access field from list of records
print(students[0]["name"])    # "Alice"
```

1. Variables, constants and data types

Programs need to store and manipulate data. **Variables** are named storage locations that hold values which can change during program execution. **Constants** are named values that remain fixed throughout the program. Every piece of data has a **data type** that determines what operations can be performed on it.

1.1 Variables

A **variable** is a named location in memory that stores a value. The value can be changed (varied) during program execution.

Declaring and assigning variables

In Python, you create a variable by assigning a value to a name:

```
# Variable declaration and assignment
score = 0
player_name = "Player1"
is_playing = True
temperature = 21.5
```

Naming rules for variables

- Must start with a letter or underscore (_)
- Can contain letters, numbers, and underscores
- Cannot contain spaces or special characters
- Cannot be a Python keyword (like `if`, `for`, `while`)
- Are case-sensitive (`Score` and `score` are different)

> **Exam tip:** Use **meaningful names** that describe what the variable stores. `player_score` is better than `x` or `ps`.

1.2 Constants

A **constant** is a named value that should not change during program execution. Constants make programs easier to read and maintain.

Why use constants?

- **Readability:** `MAX_LIVES` is clearer than `3`
- **Maintainability:** Change the value in one place only
- **Prevent errors:** Avoids accidentally changing important values

```
# Constants use UPPERCASE names
MAX_ATTEMPTS = 3
PI = 3.14159
VAT_RATE = 0.2
WELCOME_MESSAGE = "Welcome to the game"

# Use constants in calculations
total = price + (price * VAT_RATE)
```

> **Python note:** Python doesn't enforce constants — you can technically change them. The UPPERCASE naming convention signals to other programmers that the value should not be modified.

1.3 Data types

Every piece of data in a program has a **data type** that determines what values it can hold and what operations can be performed on it. Choosing the correct data type is essential for writing correct programs.

Integer (int)

Real / Float

Numbers with decimal places.

Whole numbers without decimal points.

```
age = 16
count = -5
year = 2024
```

Use for: Counting, ages, quantities, IDs

```
price = 19.99
height = 1.75
temperature = -2.5
```

Use for: Prices, measurements, averages

Boolean (bool)

Only two values: True or False.

```
found = True
game_over = False
is_valid = True
```

Use for: Yes/no decisions, flags, conditions

Character and String (str)

Characters are single symbols. Strings are sequences of characters.

```
grade = "A"           # Character
initial = "J"         # Character
name = "John Smith"   # String
message = "Hello!"    # String
empty = ""            # Empty string
```

Note: Python treats single characters as strings of length 1.

Type conversion

Convert between types using built-in functions:

```
# String to integer
age = int("16")        # 16

# String to float
price = float("9.99")  # 9.99

# Number to string
text = str(100)        # "100"
```

```
# Input is always a string!
num = int(input("Enter number: "))
```

Worked Example 1 — Declaring variables with appropriate types

Question: A program stores information about a book: its title, price, number of pages, and whether it is in stock. Declare appropriate variables for each piece of data. [4 marks]

Model answer:

```
title = "The Great Gatsby"    # String for text
price = 8.99                  # Float for decimal price
num_pages = 180               # Integer for whole number
in_stock = True               # Boolean for yes/no
```

Marking: 1 mark for each correct variable with appropriate data type.

Exam tip: Match data types to the nature of the data. Prices need decimals (float), counts are whole numbers (integer), yes/no answers are Boolean.

Worked Example 2 — Using constants

Question: A game gives players 3 lives. When a player loses a life, display how many lives remain. Use a constant for the starting lives. [3 marks]

Model answer:

```
MAX_LIVES = 3                          # Constant (1)
lives_remaining = MAX_LIVES            # Variable starts at max (1)
lives_remaining = lives_remaining - 1  # Lose one life
print("Lives remaining:", lives_remaining)  # Output (1)
```

Output: `Lives remaining: 2`

Exam tip: Constants are UPPERCASE. Using `MAX_LIVES` instead of `3` makes the code self-documenting and easier to change later.

Worked Example 3 — Type conversion from input

Question: Write code to input two numbers from the user, add them together, and display the result. [3 marks]

Model answer:

```
num1 = int(input("Enter first number: "))     # Convert to int (1)
num2 = int(input("Enter second number: "))    # Convert to int (1)
total = num1 + num2
print("Total:", total)                        # Display result (1)
```

Exam tip: `input()` always returns a string. You MUST convert to int or float before doing maths, otherwise "5" + "3" gives "53" not 8.

Worked Example 4 – Identifying data types

Question: State the most appropriate data type for each of the following: (a) A person's surname, (b) The number of items in a shopping basket, (c) Whether a user is logged in, (d) The temperature in Celsius. [4 marks]

Model answer:
(a) **String** — surnames contain text/characters **(1)**
(b) **Integer** — number of items is always a whole number **(1)**
(c) **Boolean** — logged in is either True or False **(1)**
(d) **Real/Float** — temperature can have decimal places (e.g., 21.5°C) **(1)**

Exam tip: Think about whether the value can have decimals (float), is always whole (integer), is yes/no (Boolean), or is text (string).

2. 1D lists (arrays)

A **list** (also called an array in other languages) is a data structure that stores multiple values in a single variable. Each value has an **index** that indicates its position in the list. Lists are essential when you need to work with collections of related data.

2.1 Creating lists

Lists can be created empty or with initial values:

```
# Empty list
scores = []

# List with values
names = ["Alice", "Bob", "Charlie"]
```

```
numbers = [10, 20, 30, 40, 50]
mixed = ["text", 42, True, 3.14]
```

Why use lists?

- Store multiple related values together
- Process data using loops
- Add or remove items dynamically
- Sort and search collections of data

2.2 Indexing

Each element in a list has an **index** (position number). Python uses **zero-based indexing** — the first element is at index 0.

```
List: names = ["Alice", "Bob", "Charlie", "Diana"]

Index:      0         1         2         3
          ┌─────────┬─────────┬─────────┬─────────┐
Value:    │  Alice  │   Bob   │ Charlie │  Diana  │
          └─────────┴─────────┴─────────┴─────────┘

Negative:  -4        -3        -2        -1

len(names) = 4
First item: names[0] = "Alice"
Last item:  names[3] or names[-1] = "Diana"
```

```
# Accessing elements
first = names[0]     # "Alice"
second = names[1]    # "Bob"
last = names[3]      # "Diana"
last = names[-1]     # "Diana" (negative index)

# Length of list
size = len(names)    # 4
```

2.3 List methods

Python provides built-in methods to modify lists. These are essential for Paper 2 tasks.

> **For all examples below:** Assume `names = ["Alice", "Bob", "Charlie"]` before each operation.

Method	What it does	Example	Result
`append(item)`	Add item to the end	`names.append("Diana")`	["Alice", "Bob", "Charlie", "Diana"]
`insert(index, item)`	Insert item at position	`names.insert(1, "Zoe")`	["Alice", "Zoe", "Bob", "Charlie"]
`remove(item)`	Remove first occurrence	`names.remove("Bob")`	["Alice", "Charlie"]
`pop(index)`	Remove and return item at index	`x = names.pop(0)`	x = "Alice", names = ["Bob", "Charlie"]
`pop()`	Remove and return last item	`x = names.pop()`	x = "Charlie", names = ["Alice", "Bob"]
`len(list)`	Return number of items	`len(names)`	3

2.4 Iterating through lists

You can process every element in a list using a `for` loop:

Iterate by value

```
scores = [85, 92, 78, 90]

# Process each value
for score in scores:
    print(score)

# Calculate total
total = 0
for score in scores:
    total = total + score
print("Total:", total)
```

Iterate by index

```
scores = [85, 92, 78, 90]

# Access by index
```

```
for i in range(len(scores)):
    print("Index", i, "=", scores[i])

# Modify values (needs index)
for i in range(len(scores)):
    scores[i] = scores[i] + 5
```

Exam tip: Use `for item in list` when you just need to read values. Use `for i in range(len(list))` when you need to modify values or know the position.

Worked Example 5 — Creating and populating a list

Question: Write code to input 5 test scores from the user and store them in a list. Then display all the scores. [4 marks]

Model answer:

```
scores = []                                  # Create empty list (1
for i in range(5):                           # Loop 5 times (1)
    score = int(input("Enter score: "))      # Input and convert (1
    scores.append(score)                     # Add to list

print("All scores:", scores)                 # Display list (1)
```

Exam tip: The pattern of creating an empty list, then using a loop with `append()` to populate it, is very common in Paper 2.

Worked Example 6 — Finding the maximum value

Question: Write code to find and display the highest score in a list of scores. [4 marks]

Model answer:

```
scores = [85, 92, 78, 95, 88]
highest = scores[0]                # Start with first value (1)
for score in scores:               # Check each score (1)
    if score > highest:            # Compare to current highest (1)
        highest = score            # Update if larger
print("Highest score:", highest)    # Display result (1)
```

Output: `Highest score: 95`

Exam tip: To find the maximum, start with the first element and compare each subsequent element. Update the maximum when you find a larger value.

Worked Example 7 — Calculating an average

Question: Write code to calculate and display the average of all values in a list of numbers. [4 marks]

Model answer:

```
numbers = [10, 20, 30, 40, 50]
total = 0                          # Initialise total (1)
for num in numbers:                # Loop through list (1)
    total = total + num            # Add each value
count = len(numbers)               # Get count (1)
average = total / count            # Calculate average
print("Average:", average)         # Display result (1)
```

Output: `Average: 30.0`

Exam tip: Average = total ÷ count. Use `len()` to get the count. Remember that division in Python always returns a float.

Worked Example 8 — Searching a list

Question: Write code to search for a name in a list and display whether it was found and at which position. [4 marks]

Model answer:

```
names = ["Alice", "Bob", "Charlie", "Diana"]
search = input("Enter name to find: ")
found = False                              # Flag variable (1)
position = -1
for i in range(len(names)):                # Loop with index (1)
    if names[i] == search:                 # Compare (1)
        found = True
        position = i
if found:
    print(search, "found at position", position)
```

```
else:
    print(search, "not found")              # Output result (1)
```

Exam tip: Use a Boolean flag (`found`) to track whether the search was successful. Loop with index if you need to report the position.

3. 2D lists and records

More complex data often needs more sophisticated structures. **2D lists** (two-dimensional lists) store data in rows and columns, like a table or grid. **Records** group related data fields together under named keys.

3.1 2D lists

A **2D list** is a list of lists. It's useful for representing grids, tables, or matrices.

```
# Create a 2D list (3 rows, 4 columns)
grid = [
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]
]
```

Visualising 2D lists

```
        Column: 0    1    2    3
                ┌────┬────┬────┬────┐
Row 0:          │  1 │  2 │  3 │  4 │
                ├────┼────┼────┼────┤
Row 1:          │  5 │  6 │  7 │  8 │
                ├────┼────┼────┼────┤
Row 2:          │  9 │ 10 │ 11 │ 12 │
                └────┴────┴────┴────┘

Access: grid[row][column]
grid[0][1] = 2
grid[2][3] = 12
```

3.2 Accessing 2D list elements

Use double index notation: `list[row][column]`

```
# Access individual elements
value = grid[0][1]    # Row 0, Col 1 = 2
value = grid[2][3]    # Row 2, Col 3 = 12

# Access entire row
row = grid[1]          # [5, 6, 7, 8]

# Modify an element
grid[1][2] = 99        # Change 7 to 99

# Number of rows
rows = len(grid)       # 3

# Number of columns (in first row)
cols = len(grid[0])    # 4
```

3.3 Iterating through 2D lists

Use nested loops to process every element in a 2D list:

Nested for loops (by value)

```
# Print all elements
for row in grid:
    for item in row:
        print(item, end=" ")
    print()  # New line after each row

# Output:
# 1 2 3 4
# 5 6 7 8
# 9 10 11 12
```

Nested for loops (by index)

```
# Access using indices
for r in range(len(grid)):
    for c in range(len(grid[r])):
        print(grid[r][c], end=" ")
    print()

# Calculate total of all elements
total = 0
for row in grid:
```

```
    for item in row:
        total = total + item
```

3.4 Records (dictionaries)

A **record** stores related data together using named fields. In Python, we use **dictionaries** to implement records. Each piece of data is accessed by its **key** (field name) rather than by index.

Visualising records

```
Record: student = {"name": "Alice", "age": 16, "grade": "A"}

Key:      "name"       "age"       "grade"
          ┌──────────┬──────────┬──────────┐
Value:    │  "Alice" │    16    │    "A"   │
          └──────────┴──────────┴──────────┘

Access by key (not index):
student["name"]  → "Alice"
student["age"]   → 16
student["grade"] → "A"
```

```python
# Create a record (dictionary)
student = {
    "name": "Alice Smith",
    "age": 16,
    "grade": "A",
    "enrolled": True
}

# Access fields by key
print(student["name"])     # "Alice Smith"
print(student["age"])      # 16

# Update a field
student["age"] = 17

# Add a new field
student["school"] = "High School"
```

Why use records?

- Group related data together logically
- Access data by meaningful names, not numbers

- Self-documenting — the keys describe the data
- Flexible — can add fields as needed

Records vs Lists

Feature	List	Record (dict)
Access by	Index (number)	Key (name)
Order	Ordered	Ordered (Python 3.7+)
Best for	Sequences of similar items	Related data with named fields

3.5 Lists of records

A common pattern is to store multiple records in a list — for example, a list of students or a list of products:

```
# List of records
students = [
    {"name": "Alice", "age": 16, "score": 85},
    {"name": "Bob", "age": 15, "score": 92},
    {"name": "Charlie", "age": 16, "score": 78}
]

# Access first student's name
print(students[0]["name"])    # "Alice"

# Loop through all students
for student in students:
    print(student["name"], "scored", student["score"])

# Find students with score > 80
for student in students:
    if student["score"] > 80:
        print(student["name"], "passed")
```

Worked Example 9 — Creating and accessing a 2D list

Question: A 3×3 grid stores the numbers 1—9. Write code to create this grid and display the value in the centre. [3 marks]

Model answer:

```
grid = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]                                # Create 2D list (1)
centre = grid[1][1]              # Row 1, Col 1 (1)
print("Centre value:", centre)   # Display (1)
```

Output: `Centre value: 5`

Exam tip: In a 3×3 grid, the centre is at index [1][1]. Remember: indices start at 0, so row 1 is the second row.

Worked Example 10 — Summing a 2D list

Question: Write code to calculate and display the total of all values in a 2D list of numbers. [4 marks]

Model answer:

```
grid = [
    [10, 20, 30],
    [40, 50, 60]
]
total = 0                       # Initialise total (1)
for row in grid:                # Outer loop for rows (1)
    for item in row:            # Inner loop for columns (1)
        total = total + item
print("Total:", total)          # Display result (1)
```

Output: `Total: 210`

Exam tip: Use nested loops for 2D lists — the outer loop handles rows, the inner loop handles columns within each row.

Worked Example 11 — Creating and using a record

Question: Create a record to store a product with name, price, and quantity. Display a message showing the product name and total value (price × quantity). [4 marks]

Model answer:

```
product = {
    "name": "Laptop",
    "price": 599.99,
    "quantity": 3
}                                           # Create record (1)
name = product["name"]                      # Access field (1)
```

```
value = product["price"] * product["quantity"]  # Calculate (1)
print(name, "total value: £" + str(value))   # Display (1)
```

Output: `Laptop total value: £1799.97`

Exam tip: Access dictionary values using square brackets and the key name: `record["key"]`. Keys are strings, so use quotes.

Worked Example 12 — Processing a list of records

Question: Given a list of student records, write code to find and display the name of the student with the highest score. [5 marks]

Model answer:

```
students = [
    {"name": "Alice", "score": 85},
    {"name": "Bob", "score": 92},
    {"name": "Charlie", "score": 78}
]
highest_score = students[0]["score"]    # Start with first (1)
highest_name = students[0]["name"]
for student in students:                # Loop through records (1)
    if student["score"] > highest_score:  # Compare (1)
        highest_score = student["score"]  # Update highest (1)
        highest_name = student["name"]
print("Top student:", highest_name)     # Display result (1)
```

Output: `Top student: Bob`

Exam tip: When finding the maximum in a list of records, you need to track both the maximum value AND the related data (like the name). Update both when you find a new maximum.

Worked examples summary

The table below summarises the 12 worked examples covered in this topic, organised by the type of data structure skill they assess.

Example	Skill assessed	Key technique
1	Declaring variables with types	Match data type to nature of data
2	Using constants	UPPERCASE naming, fixed values

Example	Skill assessed	Key technique
3	Type conversion from input	int() or float() on input()
4	Identifying data types	Analyse the nature of the data
5	Creating and populating lists	Empty list + loop + append()
6	Finding maximum value	Start with first, compare each
7	Calculating average	Sum all values, divide by len()
8	Searching a list	Loop with index, Boolean flag
9	Creating 2D lists	List of lists, double indexing
10	Summing 2D list	Nested loops, accumulator
11	Creating and using records	Dictionary with key-value pairs
12	Processing list of records	Loop through, access by key

Quick reference

Key definitions

Term	Definition
Variable	A named storage location in memory whose value can change during program execution
Constant	A named value that remains fixed throughout program execution (UPPERCASE by convention)
Data type	A classification that specifies what kind of value a variable can hold and what operations can be performed
Integer	A whole number without decimal places (positive or negative)
Real / Float	A number with decimal places
Boolean	A data type with only two possible values: True or False
String	A sequence of characters (text)
List (Array)	A data structure that stores multiple values in a single variable, accessed by index
Index	The position of an element in a list (starts at 0 in Python)
2D List	A list of lists, representing data in rows and columns

Term	Definition
Record	A data structure that groups related data fields together, accessed by key name (dictionary in Python)

Common list operations

Operation	Code	Description
Create empty	`mylist = []`	Creates an empty list
Create with values	`mylist = [1, 2, 3]`	Creates list with initial values
Access element	`mylist[0]`	Gets first element (index 0)
Get length	`len(mylist)`	Returns number of elements
Add to end	`mylist.append(x)`	Adds x to the end
Insert at position	`mylist.insert(i, x)`	Inserts x at index i
Remove by value	`mylist.remove(x)`	Removes first occurrence of x
Remove by index	`mylist.pop(i)`	Removes and returns item at i
Update element	`mylist[i] = x`	Changes value at index i

Type conversion summary

From	To Integer	To Float	To String	To Boolean
String "42"	`int("42")` → 42	`float("42")` → 42.0	—	`bool("42")` → True
String "3.14"	Error	`float("3.14")` → 3.14	—	`bool("3.14")` → True
Integer 42	—	`float(42)` → 42.0	`str(42)` → "42"	`bool(42)` → True
Float 3.9	`int(3.9)` → 3	—	`str(3.9)` → "3.9"	`bool(3.9)` → True
Integer 0	—	`float(0)` → 0.0	`str(0)` → "0"	`bool(0)` → False

Typical pitfalls

These are common mistakes students make in data structure questions. Avoiding these errors will help you gain marks in the exam.

Pitfall 1 — Forgetting to convert input.
Wrong: `num = input("Enter number: ")` then doing maths
Right: `num = int(input("Enter number: "))` — input() always returns a string, so you must convert before calculations.

Pitfall 2 — Index out of range.
Wrong: Accessing `mylist[5]` when the list has only 5 elements (indices 0—4)
Right: The last index is always `len(list) - 1`. A list of 5 items has indices 0, 1, 2, 3, 4.

Pitfall 3 — Confusing append and insert.
Wrong: Using `append()` when you need to add at a specific position
Right: `append(x)` adds to the end. `insert(i, x)` adds at index i.

Pitfall 4 — Wrong brackets for dictionaries.
Wrong: `student("name")` or `student{"name"}`
Right: `student["name"]` — always use square brackets to access dictionary values.

Pitfall 5 — 2D list index order.
Wrong: `grid[col][row]`
Right: `grid[row][col]` — row comes first, then column.

Pitfall 6 — int() truncates, doesn't round.
Wrong: Expecting `int(3.9)` to give 4
Right: `int()` truncates (removes decimals), so `int(3.9)` gives 3. Use `round()` if you need rounding.

Pitfall 7 — Using = instead of == in comparisons.
Wrong: `if name = "Alice":`
Right: `if name == "Alice":` — single = is assignment, double == is comparison.

Pitfall 8 — Modifying list while iterating by value.
Wrong: Trying to change values with `for item in list:`
Right: Use `for i in range(len(list)):` when you need to modify values, then use `list[i] = new_value`.

Pitfall 9 — Forgetting quotes around dictionary keys.
Wrong: `student[name]` — Python thinks name is a variable
Right: `student["name"]` — keys are strings, so use quotes.

Pitfall 10 — Empty list vs uninitialised variable.
Wrong: Using a variable without creating the list first
Right: Always initialise: `mylist = []` before using `append()` or other methods.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.